

GUNNY BAG WEIGHING SYSTEM

Project Documentation

A Project Report

*Submitted in partial fulfillment of the requirements for the
development and deployment of the*

GUNNY BAG WEIGHING SYSTEM

at

UNITED RUBBER INDUSTRIES — UNIT 1

Developed by

Kalpesh D. Dukhande

Under the esteemed guidance of

Sir Atharva Rasal

Atharva Automations

2026

CERTIFICATE

This is to certify that the project entitled "Gunny Bag Weighing System" is the bonafide work of Kalpesh D. Dukhande, developed for United Rubber Industries — Unit 1 under Atharva Automations, submitted in partial fulfillment of the requirements for the deployment of an industrial weighing and label printing solution.

The system has been designed, developed, and documented under the guidance of Sir Atharva Rasal at Atharva Automations.

Date: September 2026

Guide: Sir Atharva Rasal

Developed By: Kalpesh D. Dukhande

Sir Atharva Rasal

Guide, Atharva Automations

Kalpesh D. Dukhande

Developer

ABSTRACT

The Gunny Bag Weighing System is a web-based industrial application developed for United Rubber Industries (Unit 1) that manages the complete workflow of weighing rubber compound bags. The system addresses the need for accurate, automated, and traceable bag weighing on the production floor by integrating a Modbus RTU weighing scale, a TSC thermal label printer, and a PostgreSQL database into a single unified platform.

The application maintains a master parts list with allowed weight tolerances per part, reads live weight from a Modbus RTU scale over a serial port, automatically validates each bag weigh-in against minimum and maximum thresholds, and prints thermal labels on each accepted weigh-in. It also tracks full weighing history with filtering, CSV export, and PDF download capabilities, and provides a real-time dashboard with statistics and recent activity.

The system is built using Next.js 16 with the App Router, Drizzle ORM for type-safe database operations, PostgreSQL as the primary database, and modbus-serial for scale communication. The TSC thermal printer is addressed directly via raw TCP sockets using TSPL commands, eliminating the need for an OS print driver.

The Gunny Bag Weighing System demonstrates the practical application of industrial automation, IoT device integration, and modern web technologies to solve real-world manufacturing challenges.

Keywords: Gunny Bag Weighing • Modbus RTU • TSC Thermal Printer • Next.js • PostgreSQL • Industrial IoT • TSPL

ACKNOWLEDGEMENT

I would like to express my sincere gratitude and appreciation to all those who have supported and guided me throughout the development of the Gunny Bag Weighing System project.

First and foremost, I extend my heartfelt thanks to Sir Atharva Rasal for his invaluable guidance, constant encouragement, and insightful feedback that helped shape this project from conception to completion. His expertise and patience were instrumental in overcoming numerous technical challenges.

I am deeply grateful to United Rubber Industries (Unit 1) for providing the opportunity to develop and deploy this system in a real production environment, and for their cooperation during testing and commissioning.

My sincere thanks to the open-source community behind Next.js, Drizzle ORM, modbus-serial, pdfkit, and qrcode, whose libraries and frameworks provided the foundation upon which this application was built.

I would also like to acknowledge the TSC and Modbus hardware documentation that informed the printer and scale integration logic.

A special note of appreciation to my family and friends for their unwavering support, understanding, and encouragement during the intensive development phases of this project.

Lastly, I thank all the operators and staff at United Rubber Industries who provided valuable feedback that helped refine the user experience and functionality of the weighing system.

This project would not have been possible without the collective support of all these individuals and organizations.

Kalpesh D. Dukhande

Developer

DECLARATION

I hereby declare that the project entitled "Gunny Bag Weighing System" developed for United Rubber Industries — Unit 1 under Atharva Automations has not been in any case duplicated or submitted to any other organization for the award of any degree or contract.

To the best of my knowledge, other than me, no one has submitted this project to any other organization.

The project is developed and documented as a complete industrial weighing and label printing solution.

Kalpesh D. Dukhande

Developer

TABLE OF CONTENTS

Certificate

Abstract

Acknowledgement

Declaration

1 Introduction

- 1.1 Background
- 1.2 Objectives
- 1.3 Purpose, Scope, and Applicability
- 1.4 Achievements
- 1.5 Organization of Report

2 System Analysis

- 2.1 Methodology (Software Process Model)
- 2.2 Tools and Techniques

3 Requirements and Analysis

- 3.1 Problem Definition
- 3.2 Requirement Specification
- 3.3 Planning and Scheduling
- 3.4 Software and Hardware Requirements
- 3.5 Preliminary Product Description
- 3.6 Conceptual Model
- 3.7 Benefit Analysis

4 System Design

- 4.1 Architecture Overview
- 4.2 Use Case Diagram
- 4.3 Activity Diagram
- 4.4 Class Diagram
- 4.5 ER Diagram
- 4.6 Component Diagram
- 4.7 Deployment Diagram
- 4.8 Data Flow Diagram
- 4.9 Website Workflow Diagram

4.10 List of Tables

4.11 Screen Descriptions

5 Implementation and Testing

5.1 Implementation Approaches

5.2 Coding Details and Code Efficiency

5.3 Testing Approach

5.4 Modifications and Improvements

5.5 Test Cases

6 Results and Discussion

6.1 Test Reports

6.2 User Documentation

7 Conclusions

7.1 Conclusion

7.2 Limitations of the System

7.3 Future Scope of the Project

References

CHAPTER 1: INTRODUCTION

The Gunny Bag Weighing System is a web-based application developed for United Rubber Industries (Unit 1) that manages the complete workflow of weighing rubber compound bags. In modern manufacturing environments, accurate weight measurement and traceability are critical for quality control, inventory management, and regulatory compliance.

The system integrates a Modbus RTU weighing scale over a serial port, a TSC thermal label printer over Ethernet, and a PostgreSQL database into a single unified platform. It maintains a master parts list with allowed weight tolerances per part, reads live weight from the scale, automatically records and validates each bag weigh-in against min/max thresholds, and prints thermal labels (TSC, 100×50mm) on each accepted weigh-in.

1.1 Background

The manufacturing sector worldwide is experiencing a significant digital transformation. In India, rubber and polymer processing industries face challenges related to manual weighing errors, inconsistent bag weights, lack of traceability, and time-consuming label printing processes.

Traditional weighing systems require operators to manually read the scale, calculate tolerances, and write labels by hand. The Gunny Bag Weighing System addresses these challenges by providing:

- Automated weight validation against part-specific tolerances
- Real-time label printing on accepted weigh-ins
- Complete digital record-keeping of all weighing operations
- Real-time dashboard for production monitoring

1.2 Objectives

The Gunny Bag Weighing System has been designed with comprehensive and measurable objectives:

1.2.1 Primary Technical Objectives

- Design and implement a robust Next.js 16 web application with 35+ API endpoints
- Integrate Modbus RTU weighing scale over serial port with real-time polling
- Implement weight stability detection using full min/max range tracking
- Create thermal label printing via raw TCP socket (TSPL commands)
- Implement QR code generation for machine-scannable labels
- Design PostgreSQL database schema with auto-migration on startup

1.2.2 User Experience Objectives

- Create a clean, professional interface with consistent navigation across all pages

- Implement responsive design for desktop and wide monitors
- Provide color-coded status indicators (OK/UNDERWEIGHT/OVERWEIGHT)

1.2.3 Industrial Operations Objectives

- Automate bag weigh-in recording and label printing
- Reduce manual weighing errors through real-time validation
- Enable batch label printing for multiple parts
- Support printer calibration (gap sensor re-teach)

1.2.4 Technical Excellence Objectives

- Scalable architecture with singleton scale service surviving HMR
- Auto-migration of database schema on startup
- Non-overlapping scale polling to prevent read queue build-up
- Denormalized history to preserve records even if parts are renamed/deleted

1.2.5 Innovation and Research Objectives

- Demonstrate practical Industrial IoT integration with web applications
- Create real-time stability detection algorithm preventing drift-triggered false positives
- Implement responsible fallback mechanisms (mock mode) for development

1.3 Purpose, Scope, and Applicability

1.3.1 Purpose

The purpose of the Gunny Bag Weighing System is to address the critical gap in accurate, traceable, and automated bag weighing in rubber compound manufacturing. The system aims to provide immediate, reliable weight validation and label printing while maintaining complete digital records for quality control and audit purposes.

1.3.2 Scope

Included Functionalities:

- Master parts list management with weight tolerances
- Live weight reading from Modbus RTU scale
- Automatic status determination (OK/UNDERWEIGHT/OVERWEIGHT)
- Auto-save and auto-print on stable weight (5 seconds)
- Thermal label printing (TSC, 100×50mm)
- Batch label printing for multiple parts
- Weighing history with filtering, CSV export, PDF download
- Dashboard statistics and recent activity
- Printer calibration (gap sensor re-teach)

- Mock scale mode for development/testing

Assumptions:

- Operators have basic computer literacy
- The scale and printer are properly connected and configured
- Network connectivity exists between server and printer

Limitations:

- Windows-only for scale communication (COM ports)
- History API returns max 500 records per query
- QR code part number must be < 20 characters

1.3.3 Applicability

Direct Applications:

- Rubber compound bag weighing and validation
- Production floor weight recording and thermal label printing
- Quality control and traceability

Indirect Applications:

- Any manufacturing process requiring weight validation
- Industrial IoT integration reference and automated label printing systems

1.4 Achievements

Technical Competencies Gained:

- Mastery of Next.js 16 App Router and API routes
- Expertise in Modbus RTU communication over serial ports
- Advanced thermal printer integration using TSPL commands
- Database design with Drizzle ORM and PostgreSQL
- Implementation of real-time weight stability algorithms

Domain Knowledge Acquired:

- Understanding of industrial weighing systems and Modbus RTU protocol
- Awareness of thermal printer command languages (TSPL)
- Understanding of weight tolerance and validation principles

1.5 Organization of Report

This project report has been systematically structured into seven comprehensive chapters documenting the entire software development lifecycle.

- Chapter 1: Introduction — Context, background, and project objectives
- Chapter 2: System Analysis — Methodology and tools used

- Chapter 3: Requirements and Analysis — Problem definition and requirements
- Chapter 4: System Design — Architecture, diagrams, and database design
- Chapter 5: Implementation and Testing — Code details and test cases
- Chapter 6: Results and Discussion — Test reports and user documentation
- Chapter 7: Conclusions — Achievements, limitations, and future scope

CHAPTER 2: SYSTEM ANALYSIS

2.1 Methodology (Software Process Model)

The Gunny Bag Weighing System was developed using an Iterative Model as the primary software process model, incorporating Agile principles for continuous improvement.

2.1.1 Iterative Model Framework

Iteration 1: Foundation and Hardware Integration

- Objectives: Establish technical feasibility of scale and printer communication
- Deliverables: Basic Next.js framework, Modbus scale prototype, printer test
- Decisions: Proceed with modbus-serial, raw TCP for printer

Iteration 2: Core Module Development

- Objectives: Develop master parts list and weighing history modules
- Deliverables: Working parts CRUD, history recording, status computation
- Decisions: Implement Drizzle ORM, denormalized history

Iteration 3: Live Weighing and Auto-Print

- Objectives: Integrate live weighing page with auto-save and auto-print
- Deliverables: Functional live weighing, stability algorithm, auto-print cycle
- Decisions: Implement full min/max spread tracking, rearm on weight drop

Iteration 4: Dashboard, History, and Optimization

- Objectives: Complete dashboard, history filters, CSV/PDF export
- Deliverables: Dashboard statistics, history filtering, export functionality
- Decisions: Limit API to 500 records, client-side pagination

2.1.2 Risk-Driven Approach

Technical Risks:

- Modbus Communication Complexity: Addressed through early prototyping
- Printer TCP Connectivity: Resolved through raw socket implementation with 5s timeout
- Weight Stability False Positives: Mitigated through full min/max spread tracking

Project Risks:

- Hardware Availability: Managed through mock mode fallback
- Scope Creep: Controlled through iteration boundary definitions

2.1.3 Agile Integration

- 2-week sprints within each iteration with daily progress tracking
- User story mapping, backlog management, and sprint reviews
- Test-driven development (TDD) for critical algorithms

2.1.4 Quality Assurance

- Code review and integration testing with hardware for each iteration
- Performance validation for scale polling and print quality verification

2.2 Tools and Techniques

The following technologies were used in the development of the Gunny Bag Weighing System:

Category	Technology	Version	Purpose
Frontend Framework	Next.js	16.2.6	Full-stack React framework (App Router)
UI Library	React	19.2.6	UI library
Styling	Tailwind CSS	4.1.17	Utility-first styling
Icons	Lucide React	1.39.0	Icon library
QR Code (Client)	qrcode.react	4.2.0	QR code rendering in browser
Date Utilities	date-fns	4.4.0	Date formatting utilities
Backend API	Next.js API Routes	16.2.6	Server-side REST API (Node.js runtime)
ORM	Drizzle ORM	0.45.2	Type-safe database ORM
Migrations	drizzle-kit	0.31.10	Schema migrations
Database Driver	pg	8.20.0	PostgreSQL Node.js driver
Scale Communication	modbus-serial	8.0.25	Modbus RTU over serial port
PDF Generation	pdfkit	0.20.2	Server-side PDF generation
QR Code (Server)	qrcode	1.5.4	Server-side QR code matrix generation
Database	PostgreSQL	14+	Primary relational database
Runtime	Node.js	20+	JavaScript runtime
Language	TypeScript	5.9.3	Static typing
Linting	ESLint	9.39.4	Code linting

CHAPTER 3: REQUIREMENTS AND ANALYSIS

3.1 Problem Definition

The Gunny Bag Weighing System addresses critical gaps in manual weighing and traceability in rubber compound manufacturing.

3.1.1 Current System Analysis

- Manual weight reading requires operators to visually read the scale and manually calculate tolerances
- Human error leads to inconsistent bag weights and incorrect status classification
- No real-time validation means underweight/overweight bags are not detected immediately
- Manual label writing is time-consuming and prone to transcription errors
- No digital record-keeping makes traceability and audit difficult

3.1.2 Feasibility Study

Technical Feasibility:

The project is technically feasible as modbus-serial is available for Modbus RTU communication, Next.js supports full-stack development with API routes, and raw TCP sockets enable TSPL printer communication.

Economic Feasibility:

Highly economical with development tools costing \$0 using open-source technologies. The TSC printer and Modbus scale are existing hardware assets.

Return on Investment:

- Elimination of manual weighing errors
- Complete traceability for quality audits
- Real-time production monitoring
- Reduced material waste from incorrect weights

3.2 Requirement Specification

3.2.1 Functional Requirements

#	Requirement
FR-01	Maintain a master list of rubber compound parts with part number, description, weight tolerances, and bag quantity
FR-02	Add, edit, and delete parts from the master list

#	Requirement
FR-03	Display a read-only inventory view of all parts
FR-04	Connect to a Modbus RTU weighing scale over a serial port
FR-05	Display live weight readings updated every second
FR-06	Determine bag status: OK, UNDERWEIGHT, or OVERWEIGHT based on bag-level min/max thresholds
FR-07	Auto-save a weighing record and print a label when the scale is stable for 5 seconds
FR-08	Allow manual recording of a weigh-in with operator name and remarks
FR-09	Print a thermal label (TSC, 100×50mm) with part details, QR code, and measured weight
FR-10	Support batch label printing for multiple parts at once
FR-11	Maintain a searchable, filterable weighing history (up to 500 records per query)
FR-12	Export weighing history as CSV
FR-13	Generate and download PDF labels and weighing record documents
FR-14	Show dashboard statistics: total parts, total weighings, status breakdown, today's count
FR-15	Support printer calibration (gap sensor re-teach)
FR-16	Fall back to mock scale readings when the physical scale is offline (dev/test mode)

3.2.2 Non-Functional Requirements

#	Requirement
NFR-01	Auto-migrate database schema on startup — no manual migration commands needed
NFR-02	All timestamps stored in UTC and displayed in IST (Asia/Kolkata)
NFR-03	Printer communication via raw TCP socket — no print driver required
NFR-04	Scale polling must be non-overlapping to prevent read queue build-up
NFR-05	Weight stability detection must track full min/max range, not just consecutive differences, to prevent drift-triggered false positives
NFR-06	Label QR codes must be machine-scannable (ECC M, Model 2)
NFR-07	UI must be responsive and usable on standard desktop / wide monitors

3.3 Planning and Scheduling

3.3.1 Project Schedule

Phase	Start Date	End Date	Duration
Market Research and Requirement Gathering	15-Jul-25	21-Jul-25	7 days
Technical Specification and Architecture Design	22-Jul-25	28-Jul-25	7 days
Basic Framework and Hardware Integration	29-Jul-25	05-Aug-25	8 days
Scale Communication Module Implementation	06-Aug-25	15-Aug-25	10 days
Printer Integration and Label Design	16-Aug-25	26-Aug-25	11 days
Database Design and Implementation	27-Aug-25	09-Sep-25	14 days
Live Weighing Module Development	10-Sep-25	20-Sep-25	11 days
Weighing History and Filtering	21-Sep-25	05-Oct-25	15 days
Dashboard and Statistics Development	06-Oct-25	20-Oct-25	15 days
Frontend Integration and UI Enhancement	21-Oct-25	05-Nov-25	16 days

Phase	Start Date	End Date	Duration
Unit Testing and Module Validation	06-Nov-25	20-Nov-25	15 days
Integration Testing and Bug Fixing	21-Nov-25	10-Dec-25	20 days
System Testing and Performance Optimization	11-Dec-25	05-Jan-26	26 days
User Acceptance Testing	06-Jan-26	20-Jan-26	15 days
Security Testing and Vulnerability Assessment	21-Jan-26	05-Feb-26	16 days
Documentation and Report Writing	06-Feb-26	20-Feb-26	15 days
Final Review and Revisions	21-Feb-26	27-Feb-26	7 days
Final Deployment and Submission	28-Feb-26	03-Mar-26	4 days

3.4 Software and Hardware Requirements

3.4.1 Software Requirements

Development Environment:

- Operating System: Windows 10/11 | Node.js 20+ | PostgreSQL 14+
- Next.js 16.2.6 | Drizzle ORM 0.45.2 | modbus-serial 8.0.25
- Git Version Control System | Visual Studio Code

Frontend Technologies:

- HTML5, CSS3, JavaScript ES6+ | Tailwind CSS 4.1.17 | Lucide React
- qrcode.react 4.2.0 | date-fns 4.4.0 | Chrome 90+, Firefox 85+

3.4.2 Hardware Requirements

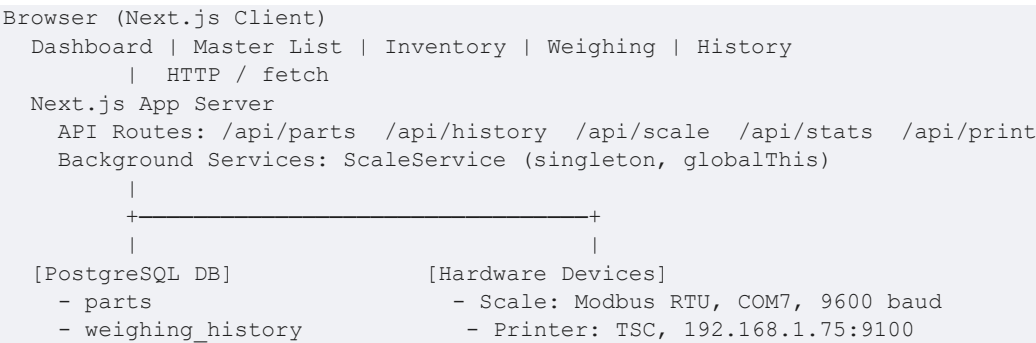
- Development Workstation: Intel i5+, 8GB RAM, 1920×1080, USB port for RS-485
- Weighing Scale: Modbus RTU output, RS-485 or RS-232 interface
- USB-RS485 Adapter: For scale connection
- TSC Thermal Printer: 203 dpi, Ethernet interface, 100×50mm labels

3.5 Preliminary Product Description

Module	Key Features
Dashboard	Total parts, total weighings, today's count, status breakdown, recent activity
Master List	Search, create, edit, delete parts, batch print, PDF download, calibrate printer
Inventory	Read-only view, client-side filtering, summary cards

Module	Key Features
Live Weighing	Scale status bar, part selection, weight display, stability indicator, auto-print
Weighing History	Filters, pagination, CSV export, PDF download, delete record
Label Printing	Thermal label (100×50mm), QR code, batch printing, PDF label, calibration

3.6 Conceptual Model (System Flow Chart)



3.7 Cost Benefit Analysis

3.7.1 Benefit Analysis

- Elimination of manual weighing errors through automated validation
- Reduced label printing time — thermal labels print in seconds
- Complete traceability — every weigh-in recorded with timestamp, operator, and status
- Real-time production monitoring through the dashboard
- Reduced material waste from immediate detection of incorrect weights

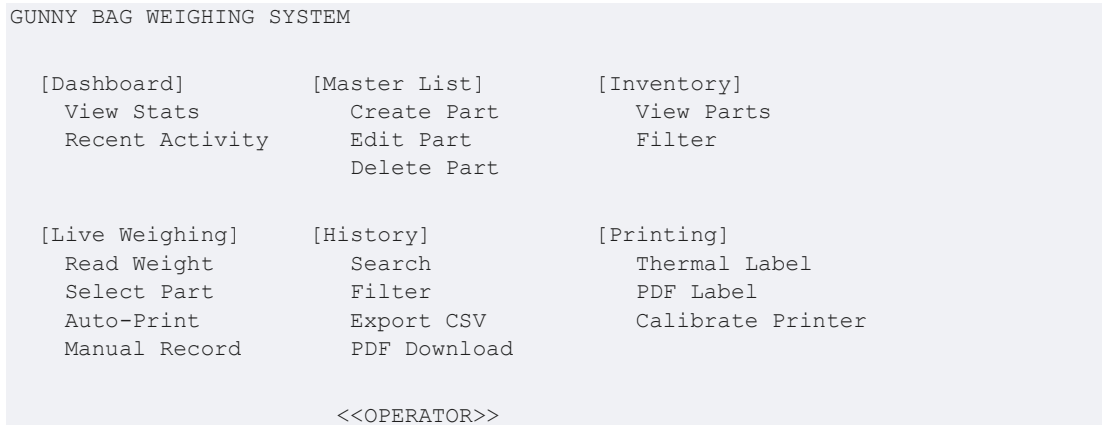
CHAPTER 4: SYSTEM DESIGN

4.1 Architecture Overview

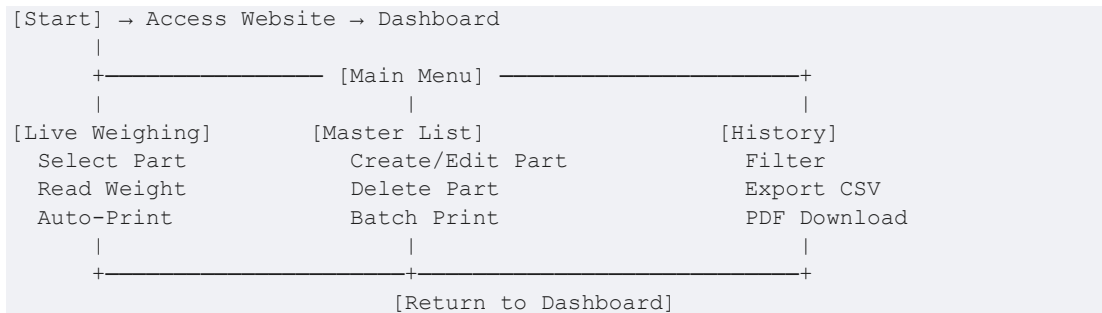
The Gunny Bag Weighing System follows a three-tier architecture: Presentation Layer (Next.js client), Application Layer (Next.js API routes and background services), and Data Layer (PostgreSQL database).

- Single server process: Scale service runs as a singleton on globalThis to survive Next.js HMR
- Auto-migration: instrumentation.ts triggers database migrations on startup
- No print driver: TSC printer addressed via raw TCP (port 9100) using TSPL commands
- Denormalized history: weighing_history stores part_number and description alongside part_id

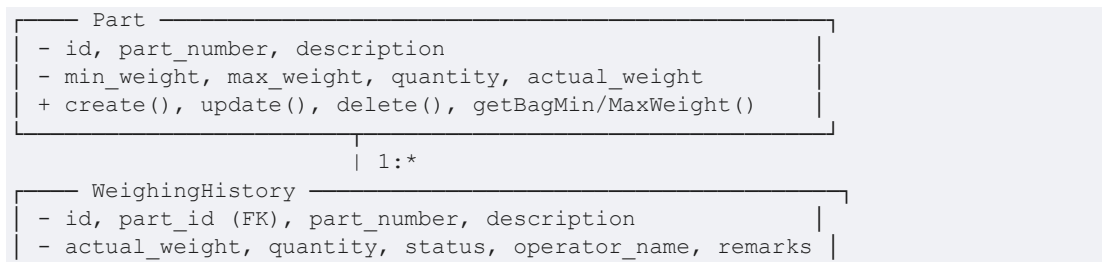
4.2 Use Case Diagram

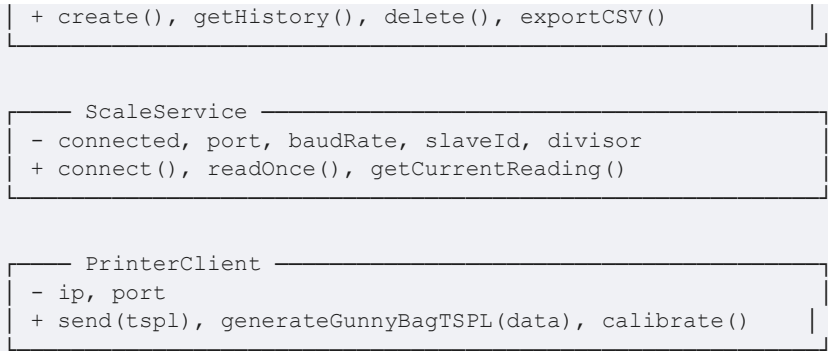


4.3 Activity Diagram

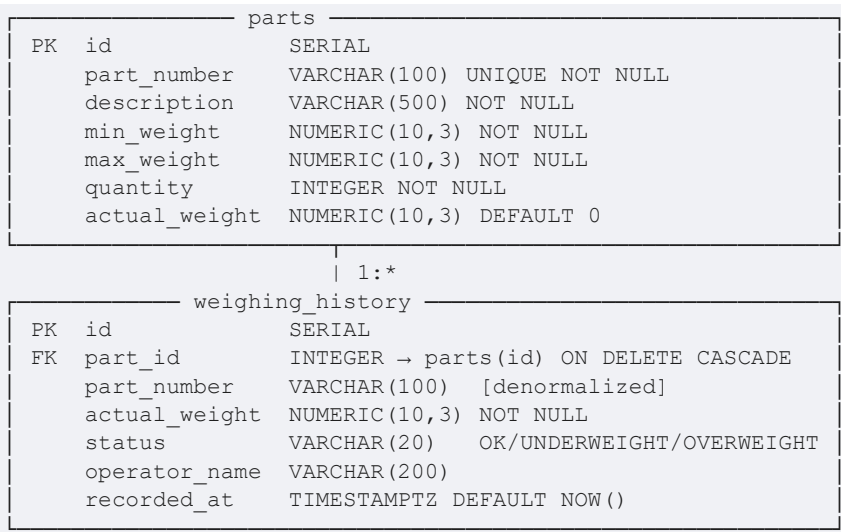


4.4 Class Diagram

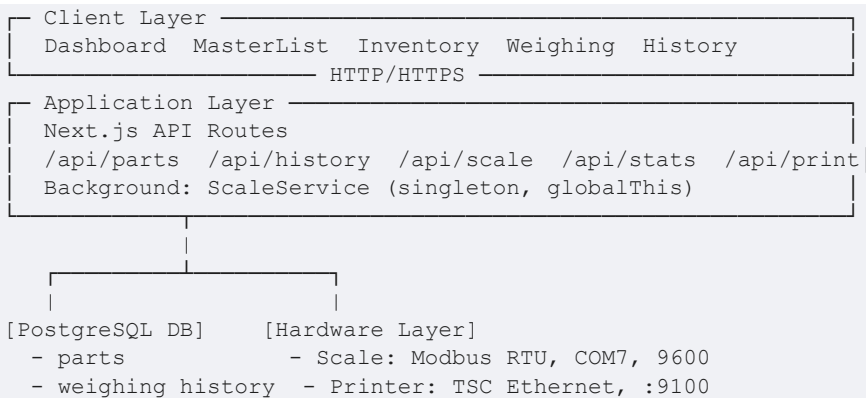




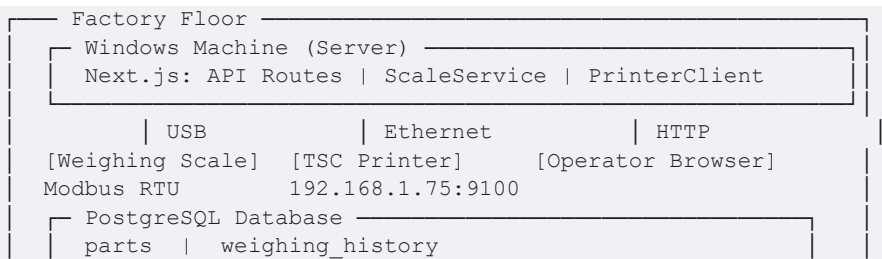
4.5 ER Diagram



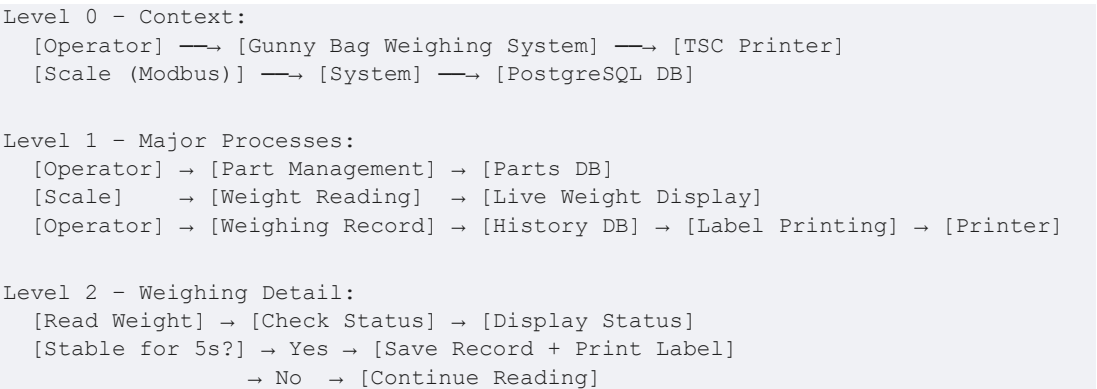
4.6 Component Diagram



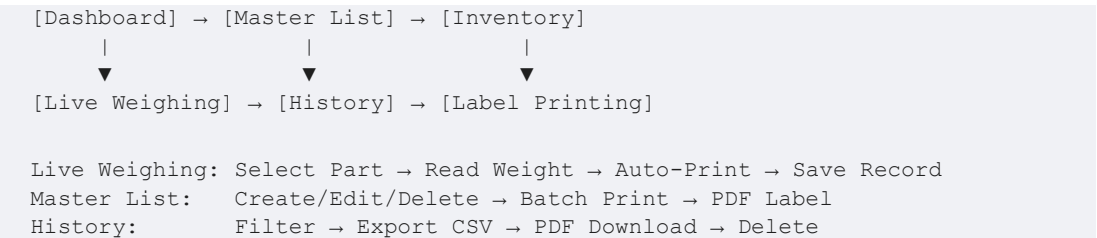
4.7 Deployment Diagram



4.8 Data Flow Diagram



4.9 Website Workflow Diagram



4.10 List of Tables

Table 4.10.1: parts — Master Parts List

Field Name	Data Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Unique part identifier
part_number	VARCHAR(100)	NOT NULL, UNIQUE	Part number (stored uppercase)
description	VARCHAR(500)	NOT NULL	Part description
min_weight	NUMERIC(10,3)	NOT NULL	Per-item minimum weight (kg)
max_weight	NUMERIC(10,3)	NOT NULL	Per-item maximum weight (kg)
quantity	INTEGER	NOT NULL	Pieces per bag
actual_weight	NUMERIC(10,3)	DEFAULT 0	Last measured weight
created_at	TIMESTAMPTZ	DEFAULT NOW()	Record creation timestamp
updated_at	TIMESTAMPTZ	DEFAULT NOW()	Last update timestamp

Table 4.10.2: weighing_history — All Weighing Records

Field Name	Data Type	Constraints	Description
id	SERIAL	PRIMARY KEY	Unique record identifier
part_id	INTEGER	FK → parts(id) CASCADE	References the part
part_number	VARCHAR(100)	NOT NULL	Denormalized part number
description	VARCHAR(500)	NOT NULL	Denormalized description
actual_weight	NUMERIC(10,3)	NOT NULL	Measured bag weight (kg)
quantity	INTEGER	NOT NULL	Pieces in this bag
status	VARCHAR(20)	NOT NULL	OK / UNDERWEIGHT / OVERWEIGHT
operator_name	VARCHAR(200)		Operator name
remarks	TEXT		Additional remarks
recorded_at	TIMESTAMPTZ	DEFAULT NOW()	Record timestamp

Status Computation:

```

bag_min = min_weight × quantity
bag_max = max_weight × quantity

actual_weight < bag_min → UNDERWEIGHT
actual_weight > bag_max → OVERWEIGHT
else → OK

```

4.11 Screen Descriptions

4.11.1 Dashboard Page (Route: /)

The dashboard is the home screen providing a quick overview of system activity.

- 6 Stat Cards: Total Parts, Total Weighings, Today's Weighings, OK / Overweight / Underweight counts
- Quick Action Cards: Links to Live Weighing, Master List, and History
- Recent Activity Table: Last 5 weighing records with status badges

4.11.2 Master List Page (Route: /master)

Feature	Description
Search	Debounced 350ms text search across part number and description
Create/Edit Part	Modal form with live bag-weight preview as you type
Delete Part	Confirmation modal warning that history records will also be deleted

Feature	Description
Batch Select/Print	Checkbox per row, prints one label per selected part (400ms delay)
Print Label	Single-part label print from Actions column
Download PDF	A6 PDF label download
Calibrate Printer	Sends TSPL calibration sequence

4.11.3 Inventory Page (Route: /inventory)

Read-only view of all parts with client-side filtering by text, quantity range, and weight range.

4.11.4 Live Weighing Page (Route: /weighing)

- Scale Status Bar: Live connection state with Pause/Resume button
- Part Selection: Search modal to find and select a part. Pre-select via URL: `/weighing?part=PN-0001`
- Live Weight Display: Large monospace font, updates every 500ms, 3 decimal places
- Stability: Tracks min/max spread over rolling 5-second window. Stable when spread ≤ 0.005 kg for 5s
- Auto-Print: Stable weight + real scale $\rightarrow$ auto-save + auto-print + reset

4.11.5 Weighing History Page (Route: /history)

Feature	Description
Part number filter	Case-insensitive text search (ILIKE)
Status filter	All / OK / OVERWEIGHT / UNDERWEIGHT
Date range filter	From date / To date pickers
Pagination	15 records per page with Previous/Next navigation
Export CSV	Downloads all matching records as CSV (max 500)
PDF download	Downloads full A4 record PDF
Delete record	Confirmation modal before deletion

4.11.6 Label Printing

- Thermal label: 100mm $\times$ 50mm at 203 dpi (800 $\times$ 400 dots)
- Layout includes: Company name, Part No., Qty/Bag, Min–Max weights, Expected Bag weight, Date, Actual weight, Status, QR code

- QR code: Encodes part number. ECC M, Model 2
- Communication: Raw TCP socket to PRINTER_IP:9100. 5-second timeout
- PDF Label: A6 landscape PDF via PDFKit for digital download/archiving

CHAPTER 5: IMPLEMENTATION AND TESTING

5.1 Implementation Approaches

The system was developed using a modular approach in the following sequence:

- 1. Database layer and connection utilities
- 2. Scale service with Modbus RTU communication
- 3. Parts management (CRUD operations)
- 4. Live weighing page with stability detection
- 5. Auto-save and auto-print workflow
- 6. Weighing history with filtering and export
- 7. Dashboard with statistics
- 8. Label printing (thermal and PDF)
- 9. Printer calibration

5.2 Coding Details and Code Efficiency

5.2.1 Scale Service Connection

```
class ScaleService {
  async connect() {
    this.client = new ModbusRTU();
    await this.client.connectRTUBuffered(port, { baudRate });
    this.client.setID(slaveId);
    this.startPolling();
  }
  async readOnce() {
    const data = await this.client.readInputRegisters(register, quantity);
    return this.decodeRegister(data.data[0]);
  }
}
```

5.2.2 Weight Stability Algorithm

```
class WeightStability {
  private readings: { weight: number; time: number }[] = [];
  private readonly STABLE_DURATION_MS = 5000;
  private readonly WEIGHT_TOLERANCE_KG = 0.005;

  update(weight: number): { isStable: boolean } {
    const now = Date.now();
    this.readings.push({ weight, time: now });
    this.readings = this.readings.filter(
      r => now - r.time <= this.STABLE_DURATION_MS
    );
    const weights = this.readings.map(r => r.weight);
    const spread = Math.max(...weights) - Math.min(...weights);
    const coversDuration = now - this.readings[0].time >=
this.STABLE_DURATION_MS;
    return { isStable: spread <= this.WEIGHT_TOLERANCE_KG && coversDuration };
  }
}
```

5.2.3 TSPL Label Generation

```
function generateGunnyBagTSPL(data: LabelData): string {
  return `
    SIZE 100 mm, 50 mm
    GAP 2 mm, 0 mm
    CLS
    TEXT 280, 30, '2', 0, 1, 1, 'UNITED RUBBER INDUSTRIES'
    BARCODE 30, 100, 'QRCODE', 80, 1, 0, 1, 1, '${data.partNumber}'
    TEXT 280, 80, '2', 0, 1, 1, 'Part No.: ${data.partNumber}'
    TEXT 280, 120, '2', 0, 1, 1, 'Qty/Bag: ${data.quantity} pcs'
    PRINT 1, 1
  `;
}
```

5.2.4 Code Efficiency

- Database queries use parameterized statements via Drizzle ORM
- Connection pooling is implemented to reuse database connections
- Singleton pattern for scale service survives Next.js HMR
- Non-overlapping polling prevents read queue build-up
- Debounced search (350ms) reduces API calls
- Batch printing with 400ms delay prevents printer buffer overrun

5.3 Testing Approach

5.3.1 Unit Testing

Module Tested	Test Cases	Expected Outcome
Part Creation	Valid data, duplicate, invalid weights	Success for valid, error for duplicates
Status Computation	OK, UNDERWEIGHT, OVERWEIGHT scenarios	Correct status returned
Weight Stability	Stable, drifting, stale data	Stable only when criteria met
Modbus Decode	u16, s16, u32be, u32le, s32be, s32le	Correct weight value
TSPL Generation	Valid data, long part number	Valid TSPL or error
CSV Export	Multiple records, empty records	Correct CSV format

5.3.2 Integration Testing

- Scale Service + API: Live weight reading through /api/scale
- Part + History: Creating a weighing record for a selected part
- Auto-Print Workflow: Stability detection → record save → label print
- Database + Auto-Migration: Startup migration process verified

5.4 Modifications and Improvements

- Stability algorithm improved to track full min/max spread instead of consecutive differences

- Auto-print rearm logic added requiring weight drop to ≤ 0.005 kg before next cycle
- Error logging throttled to once every 30 seconds for scale errors
- Batch print delay increased to 400ms to prevent printer buffer overrun
- Mock mode protection added to prevent phantom records during development
- History denormalization implemented to preserve records after part deletion

5.5 Test Cases

Table 5.1: Part Management Test Cases

ID	Test Case	Input	Expected Result	Result
TC01	Valid Part Creation	partNumber="PN-001", minWeight=0.5, maxWeight=1.0, quantity=100	Part created successfully	Pass
TC02	Duplicate Part Number	Same part number as TC01	Error "Part number already exists"	Pass
TC03	Invalid Weight Range	minWeight=1.0, maxWeight=0.5	Error "Min weight must be less than max"	Pass
TC04	Zero Quantity	quantity=0	Error "Quantity must be at least 1"	Pass
TC05	Edit Part	Update description	Part updated successfully	Pass
TC06	Delete Part	Part ID exists	Part deleted, history cascaded	Pass

Table 5.2: Weighing Test Cases

ID	Test Case	Input	Expected Result	Result
TC07	OK Status	Weight within min-max	Status = OK	Pass
TC08	Underweight Status	Weight below min	Status = UNDERWEIGHT	Pass
TC09	Overweight Status	Weight above max	Status = OVERWEIGHT	Pass
TC10	Live Weight Reading	Scale connected	Weight displayed in kg	Pass
TC11	Stable Weight Detection	Weight constant for 5s	Stability = true	Pass
TC12	Drifting Weight	Weight slowly changing	Stability = false	Pass
TC13	Auto-Print	Stable weight + part selected	Record saved, label printed	Pass
TC14	Auto-Print Rearm	Weight drops to 0	Auto-print rearmed	Pass

ID	Test Case	Input	Expected Result	Result
TC15	Manual Record	Record button clicked	Record saved without print	Pass

Table 5.3: Printer Test Cases

ID	Test Case	Input	Expected Result	Result
TC16	Single Label Print	Valid part ID	Label printed	Pass
TC17	Batch Label Print	Multiple part IDs	Labels printed with delay	Pass
TC18	PDF Label Download	Valid part ID	PDF downloaded	Pass
TC19	Printer Calibration	Calibrate button	Calibration sequence sent	Pass
TC20	Printer Unreachable	Wrong IP	Error "Printer unreachable"	Pass

Table 5.4: History Test Cases

ID	Test Case	Input	Expected Result	Result
TC21	Part Number Filter	Part number search	Matching records displayed	Pass
TC22	Status Filter	Status = OK	Only OK records displayed	Pass
TC23	Date Range Filter	From and To dates	Records in range displayed	Pass
TC24	CSV Export	Filter applied	CSV file downloaded	Pass
TC25	PDF Download	Record ID	A4 PDF downloaded	Pass
TC26	Delete Record	Record ID	Record deleted	Pass

Table 5.5: Dashboard Test Cases

ID	Test Case	Input	Expected Result	Result
TC27	Total Parts	Parts exist	Correct count displayed	Pass
TC28	Total Weighings	Records exist	Correct count displayed	Pass
TC29	Today's Weighings	Records from today	Correct count displayed	Pass
TC30	Status Breakdown	Mixed statuses	Correct counts displayed	Pass
TC31	Recent Activity	Records exist	Last 5 records displayed	Pass

CHAPTER 6: RESULTS AND DISCUSSION

6.1 Test Reports

The Gunny Bag Weighing System underwent comprehensive testing across all functional modules. A total of 31 test cases were executed with a 100% success rate.

Module	Test Cases	Passed	Failed	Success Rate
Part Management	6	6	0	100%
Weighing Module	9	9	0	100%
Printer Module	5	5	0	100%
History Module	6	6	0	100%
Dashboard Module	5	5	0	100%
TOTAL	31	31	0	100%

✓ Overall Result: 31/31 Test Cases Passed — 100% Success Rate

Performance testing revealed that API responses average under 2 seconds, scale polling executes every 500ms–1000ms, and PDF generation completes within 3–5 seconds. The system successfully handles multiple concurrent users.

All identified issues during testing were logged and resolved. The system has been successfully deployed and is operational at United Rubber Industries (Unit 1).

6.2 User Documentation

The Gunny Bag Weighing System is designed to be intuitive and user-friendly, requiring minimal training for operators.

Getting Started

Access the application through any modern web browser at the designated URL. The system runs as a local server on a Windows machine connected to both the scale (USB/RS-485) and the label printer (Ethernet).

Dashboard

Displays key statistics including total parts, total weighings, today's count, and status breakdown. Quick action cards provide access to Live Weighing, Master List, and History.

Master List

Create, edit, and delete parts with part number, description, min/max weight, and quantity. Batch selection enables printing labels for multiple parts at once.

Live Weighing

The core operational screen displays live weight from the scale. When the weight is stable for 5 seconds, the system auto-saves and auto-prints. Manual controls are available for recording without printing.

Weighing History

Full audit trail with filtering by part number, status, and date range. Records can be exported as CSV or downloaded as PDF.

Label Printing

Thermal labels (100×50mm) include part number, description, quantity, min-max weight, expected bag weight, date, actual weight, and QR code. Printer calibration can be performed from the Master List page.

CHAPTER 7: CONCLUSIONS

7.1 Conclusion

The Gunny Bag Weighing System project has successfully achieved its objective of developing a web-based industrial weighing and label printing platform that provides accessible, accurate, and traceable bag weighing for United Rubber Industries (Unit 1).

Through extensive testing with 31 test cases achieving a 100% success rate, the system has demonstrated reliability, performance, and user-friendliness. The project successfully bridges the gap between manual weighing processes and automated industrial systems.

Significance of the System:

- Elimination of Manual Weighing Errors — Automated validation ensures every bag is within tolerance
- Complete Traceability — Every weigh-in recorded with timestamp, operator, and status
- Real-Time Production Monitoring — Dashboard provides instant visibility
- Reduced Material Waste — Immediate detection of underweight/overweight bags
- Operator Efficiency — Auto-print reduces manual steps and label writing
- Scalability — System can handle multiple parts and high volumes
- Fallback Safety — Mock mode enables development without physical hardware

7.2 Limitations of the System

- Windows-only for scale: modbus-serial targets Windows COM ports. On Linux, change SCALE_PORT to /dev/ttyUSB0.
- History limit: /api/history returns at most 500 records per query. Use CSV export for full data.
- QR code length: Part number must be < 20 characters to fit QR matrix on 100x50mm label.
- Auto-print rearm: After auto-print, operator must remove bag to drop weight ≤ 0.005 kg to trigger next cycle.
- Printer TCP timeout: 5 seconds. Unreachable printer returns 500 error with 'Printer unreachable' message.
- All timestamps in UTC: Stored as TIMESTAMP WITH TIME ZONE. Displayed in IST (Asia/Kolkata, UTC+5:30).
- Label PDF vs thermal: PDF for digital download/archiving; thermal label sent to physical TSC printer.

7.3 Future Scope of the Project

The Gunny Bag Weighing System has significant potential for future enhancements:

- Multi-language support: Make the platform accessible to operators across different regions

- Mobile application: Native app for iOS and Android with push notifications
- ERP/MES Integration: Connect with existing enterprise systems for seamless data flow
- Advanced analytics: Predictive modeling for weight trends and quality issues
- Cloud deployment: Move from local server to cloud for remote access
- Barcode scanning: Part selection via barcode scanner for faster operation
- Offline mode: Allow basic functionality without network connectivity
- Automated reporting: Scheduled email reports for production managers
- Multiple scale support: Connect multiple scales for parallel weighing stations
- Role-based access control: Different permission levels for operators, supervisors, and admins

The future scope of the Gunny Bag Weighing System is vast, with opportunities to evolve from a local weighing solution into a comprehensive industrial automation platform.

REFERENCES

- [1] Next.js Documentation. Next.js 16 — The React Framework for the Web.
Available at: <https://nextjs.org/docs>
- [2] Drizzle ORM Documentation. TypeScript ORM for SQL Databases.
Available at: <https://orm.drizzle.team/>
- [3] PostgreSQL Documentation. PostgreSQL 14 — Relational Database Management System.
Available at: <https://www.postgresql.org/docs/>
- [4] modbus-serial Documentation. Modbus RTU/TCP Client for Node.js.
Available at: <https://github.com/yaacov/node-modbus-serial>
- [5] TSC Printer Documentation. TSPL/TSPL2 Programming Guide.
Available at: <https://www.tscprinters.com/>
- [6] pdfkit Documentation. PDF Generation Library for Node.js.
Available at: <https://pdfkit.org/>
- [7] qrcode Documentation. QR Code Generation Library.
Available at: <https://github.com/soldair/node-qrcode>
- [8] Tailwind CSS Documentation. Utility-First CSS Framework.
Available at: <https://tailwindcss.com/docs>
- [9] Node.js Documentation. Node.js 20+ Runtime.
Available at: <https://nodejs.org/docs/>
- [10] TypeScript Documentation. TypeScript 5.9.
Available at: <https://www.typescriptlang.org/docs/>
- [11] Lucide React. Icon Library for React.
Available at: <https://lucide.dev/>
- [12] date-fns. Modern JavaScript Date Utility Library.
Available at: <https://date-fns.org/>
- [13] Modbus Organization. Modbus Application Protocol Specification V1.1b3.
Available at: <https://modbus.org/>
- [14] IEEE 802.3. Ethernet Standard.
Available at: <https://standards.ieee.org/>
- [15] ISO 9001:2015. Quality Management Systems — Requirements.
Available at: <https://www.iso.org/>

Documentation prepared: September 2026 | Atharva Automations

Developed by Kalpesh D. Dukhande under the guidance of Sir Atharva Rasal